Long-Context Underwater Video Understanding

Tracking objects and identifying temporal patterns across long underwater video sequences

Project plan: the problem, our approach, what already exists, and what we must solve to make it our own

Official problem statement

"Long-context underwater video understanding - tracking objects and identifying temporal patterns across long underwater video sequences."

Our approach in one line: break long underwater videos into manageable pieces of work, **without breaking** the memory of what happened, which object is which, or the patterns that build up over time.

How to read the labels in this document
[Verified] - checked by web search in November 2026 (link given). Still confirm licence and numbers on the official page.
[Not re-checked] - well-known tool, but not re-verified for this document.
[Established] - standard, well-known technique. **[Needs adapting]** - exists but must be changed for underwater use.
[Our idea] - a proposal that has not been tested yet.

Date: 12 November 2026

Contents

1. The problem statement and what it asks for
2. Our approach: what we are building
3. System design (diagram)
4. Temporal patterns: the second half of the problem
5. What is available on the internet right now
6. What we can build today
7. Problems we must solve (this makes it ours)
8. Our new merged approach
9. How we prove it works
10. Build plan, step by step
11. Hardware and costs
12. Risks and honest limits
13. Sources

1. The problem statement and what it asks for

Official problem statement

> *"Long-context underwater video understanding - tracking objects and identifying temporal patterns across long underwater video sequences."*

Breaking the statement into parts

Part of the statement	What it means	What the system must deliver
Long-context	Videos last hours, not seconds. The system must remember what happened earlier.	Memory that survives the whole video; no reset when the video is split for processing.
Underwater	Colour loss, low contrast, particles, poor light, occlusion by coral and rocks.	Models that work on murky footage, and that know when visibility is too bad to trust.
Video understanding	Not just boxes: what is happening and why it matters.	Events, summaries and searchable descriptions.
Tracking objects	Follow each fish, animal or object through time.	Detection + IDs that stay the same over time, including after it hides and comes back.
Identifying temporal patterns	Find what repeats, what changes and what is unusual over time.	Cycles, trends, recurring visits, typical movement routes, change points and anomalies (section 4).

Why this is hard today

- Most AI systems look at one frame or a short clip at a time, so they **forget** what came before.
- A fish hides behind coral or in murky water and comes back with a **new ID**, so it is counted many times.
- Many fish of the same species **look almost identical**, so identities get swapped.
- Patterns only appear over **long time spans**, but long videos are too big to feed to a model at once.
- Wrong IDs create **fake patterns** (a fish 'visiting' ten times is really ten tracking breaks).
- Simple alerts cannot tell a **real unusual event** from a **tracking mistake** or bad visibility.

Who needs it

Marine biologists, fish farms and underwater inspection teams need correct counts, movement histories, behaviour patterns and searchable archives. Watching hours of footage by hand does not scale.

Our research question (derived from the statement)

> **How can we track objects and find temporal patterns across a long underwater video, while still processing it efficiently in smaller pieces - without losing context or identity?**

Note: the problem statement does not say *how* to solve it. Splitting the video into clips, three-level memory and using a vision-language model are **our approach**, described from section 2 onwards.

2. Our approach: what we are building

You upload a long underwater video (or connect a camera). The system returns:

#	Output	What it means for the user
1	Annotated video	The same video, same length and frame rate, with a box and ID on every animal or object.
2	Track table	For each object: class, when it appeared, its path, when it left.
3	Persistent IDs	The same fish keeps its ID after short disappearances. If the system is not sure, it says **uncertain** instead of guessing.
4	Event timeline	Timestamped events: arrived, left, came back, group formed, visibility dropped.
5	Summaries	Short text for each part of the video and for the whole video, written by a vision-language model.
6	Alerts	Unusual crowding, sudden emptiness, odd movement, unexpected objects.
7	Search	Search by fish ID, time, event type or plain text ("when did many fish gather?").
8	Dashboard	A web page to play the video, see the timeline and read summaries and alerts.

Three kinds of memory

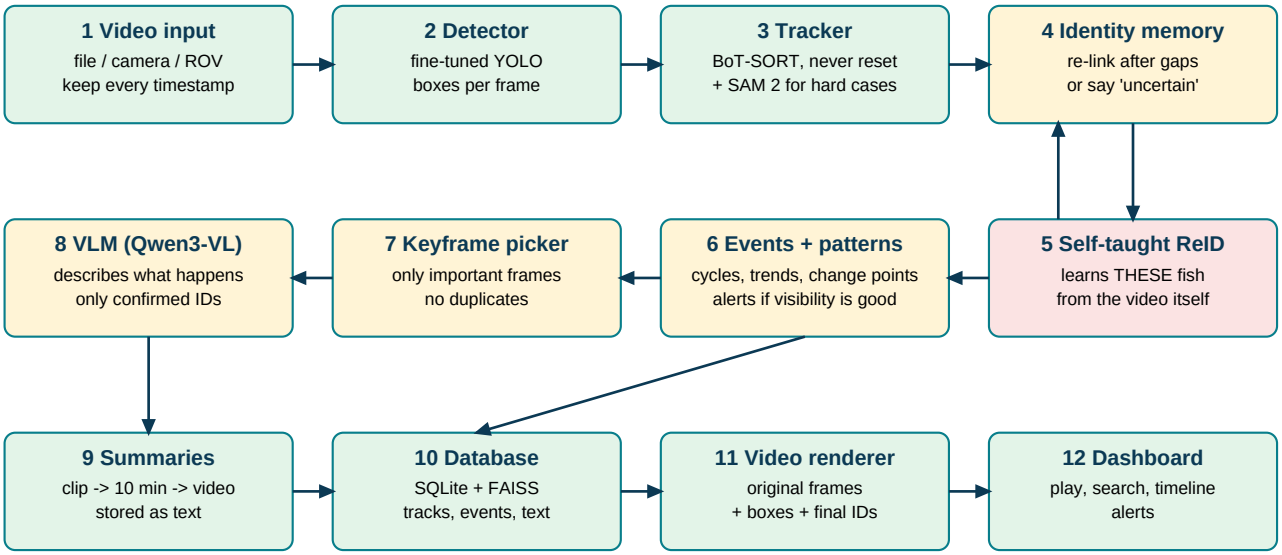
Memory	Holds	Lives for
A. Short-term (tracking)	Recent boxes, speed, appearance, current IDs, lost tracks	Seconds to minutes (in RAM)
B. Persistent (objects and events)	Global identities, paths, first/last seen, re-identification history, events	The whole video (SQLite database)
C. Semantic (meaning)	Clip summaries, activity descriptions, video summary, search vectors	The whole archive (SQLite + FAISS)

Important design rule: clips are only a unit of work (saving, checkpoints, VLM windows). The tracker and the memory run continuously and **never restart at a clip boundary**. Sending frames as network "packets" is not needed on one machine; in-memory batches with a frame number and timestamp are enough.

How our approach covers the problem statement

Statement asks for	Our approach	Section
Long-context	Continuous tracker + three-level memory; clips are only work units	2, 3
Tracking objects	YOLO + BoT-SORT + identity memory + self-taught re-ID (STIM)	3, 8
Identifying temporal patterns	Time series from tracks, cycles, change points, route clustering, revisits	4
Video understanding	Keyframes + VLM summaries grounded in tracker facts	3
Underwater conditions	Fine-tuning on underwater data, visibility score, health-gated alerts	7

3. System design



Memory A (live tracks, in RAM) -> Memory B (identities & events, SQLite) -> Memory C (summaries, FAISS)

Clips are only work units for saving and the VLM. The tracker and memory never restart at a clip boundary.

How data flows

- **Sequential:** the tracker and identity memory must see frames in order (one per camera).
- **Parallel:** detection in batches, the VLM in a separate worker, and video rendering.
- **Failures:** everything is saved by frame number. After a crash we resume from the last checkpoint. If the VLM fails, tracking keeps going.
- **Correct output video:** we re-read the original video and draw the final IDs on every original frame, matched by frame number - never rebuilt from sampled frames.

4. Temporal patterns: the second half of the problem

Tracking tells us *where each object is*. Temporal patterns tell us *what keeps happening over time*. The problem statement asks for both. Patterns are found from the **stored track and event data** (memory B), not by re-watching the video, so they are cheap to compute even for very long recordings.

Kinds of patterns we look for

Pattern	Underwater example	Needs reliable IDs?
Cycles (periodic)	Fish activity rises at dawn and dusk; activity follows the tide	No - counts are enough
Trends	Number of fish slowly falls over a week	No
Change points	Sudden switch from calm to very active; visibility collapse	No
Typical routes	Fish often patrol the reef edge, or swim through the same gap	Partly - short tracks are enough
Recurring visits	The same turtle comes back every ~40 minutes	**Yes** - needs identity across gaps
Event sequences	A predator appears, then the school tightens, then disperses	Partly
Anomalies	Unusual crowding, unknown object, sudden emptiness	No, but must check tracker health

How we find them

Step	Method	Status
1. Turn tracks into time series	Per minute: count per class, mean speed, spread of the group, time in each zone, arrivals/departures	[Established]
2. Cycles	Autocorrelation, Fourier transform, Lomb-Scargle (works with gaps in the data)	[Established]
3. Change points	PELT / binary segmentation, e.g. the ruptures Python library	[Established, library not re-checked]
4. Typical routes	Cluster trajectories (dynamic time warping + HDBSCAN) per camera zone	[Established, needs adapting]
5. Recurring visits	Revisit intervals per global ID from the identity memory	[Our idea] - uses only confirmed/probable IDs
6. Event sequences	Count which events usually follow which (simple sequence mining)	[Established]
7. Anomalies	Deviation from the learned normal pattern, ignored when visibility or tracker health is bad	[Our idea]
8. Describe in words	VLM turns detected patterns + keyframes into plain-language explanations	[Needs adapting]

What is new here [Our idea]

- **Uncertainty-aware patterns:** because our identity memory marks links as confirmed / probable / uncertain, pattern statistics can be reported as a range (e.g. '3 to 5 revisits') instead of one possibly wrong number.
- **Patterns separated from tracking errors:** a 'pattern' that matches a burst of new IDs or a visibility drop is flagged as a likely tracking artefact.
- **The VLM explains, it does not discover:** statistics find the pattern; the VLM describes it. This avoids invented patterns.

Data gap: daily or tidal cycles need recordings of many hours or days from a fixed camera. Public tracking datasets are short, so we need long recordings (our own or open archives) and we can test methods by **planting known patterns** into real data.

5. What is available on the internet right now

Models and tools

Job	Tool	Status / notes	Link
Read / write video	FFmpeg, PyAV, OpenCV	Free, open source. [Not re-checked]	ffmpeg.org
Detect objects	Ultralytics YOLO26 (or YOLO11)	[Verified] NMS-free, edge-friendly. Licence AGPL-3.0 or paid Enterprise.	docs.ultralytics.com
Detect (alternative)	RT-DETR	Transformer detector, heavier. [Not re-checked]	-
Short-term tracking	BoT-SORT, ByteTrack	Built into Ultralytics. BoT-SORT handles camera motion (ROV). [Not re-checked]	-
Fish-specific tracker	SU-T	[Verified] Unscented Kalman filter + fish-shaped IoU. 34.1 HOTA / 44.6 IDF1 on MFT25.	vranlee.github.io/SU-T
Schooling-fish tracker	TIDE	[Verified, partly] Only an aggregator listing found; read the paper first.	vranlee.github.io/TIDE
Mask tracking (video)	SAM 2 / SAM 2.1 (Meta)	[Verified] Click or box on one frame, it follows the object as a pixel mask through the video. Apache 2.0. Sizes from tiny (38.9M params) to large (224.4M).	github.com/facebookresearch/sam2
Long-video mask tracking	SAM2Long	[Verified] ICCV 2025. Training-free memory tree on top of SAM 2; fewer drifting errors in long videos.	github.com/Mark12Ding/SAM2Long
Segment / track by text	SAM 3 (Meta)	[Verified] Text prompt like "fish" finds and tracks all fish. Heavy; use for labelling and hard cases.	github.com/facebookresearch/sam3
Video features	V-JEPA 2 / 2.1 (Meta)	[Verified] Self-supervised video encoder. 2.1 has dense features; small 80M/300M versions.	github.com/facebookresearch/vjepa2
Vision-language model	Qwen3-VL (2B/4B/8B/32B + MoE)	[Verified] Long context, timestamp alignment. Main choice for summaries.	github.com/QwenLM/Qwen3-VL
Vision-language (alt.)	InternVideo2.5 / InternVideo3	[Verified] 2.5 is released. InternVideo3 paper exists; code release not confirmed.	github.com/OpenGVLab/InternVideo
Smart frame selection	VideoBrain	[Verified] ICML 2026. Uses 30-40% fewer frames. We borrow the idea.	arXiv 2602.04094
Self-taught identity	new idtracker.ai	[Verified] Learns individual identities from the video itself with contrastive learning (lab videos).	eLife preprint
Global track linking	Min-cost flow, GMCP, CoNo-Link	[Verified] Classic and newer methods to link track pieces over a whole video.	arXiv 2312.08951
Storage / search	SQLite, FAISS	Free. Enough for a prototype. [Not re-checked]	-
Accuracy scoring	TrackEval	HOTA, IDF1, ID switches. [Not re-checked]	-
App	FastAPI, Streamlit	Free. [Not re-checked]	-

Datasets

Dataset	What is inside	Track IDs?	Licence	Good for
SFISHTRACK	Balearic Sea videos. Secondary source: 54 videos, 24,084 frames, 1080p, 5-40 m deep.	Yes + masks	CC BY 4.0 [Verified]	Closest to real open water. Start here.
MFT25	15 videos, 48,066 frames, 408,578 boxes (groupers, koi).	Yes	Check (download may need password)	Tracker / ReID training. Fish tanks, not open sea.
BrackishMOT	98 sequences, murky brackish water, small fish schools.	Yes	CC BY 4.0 (unconfirmed)	Turbid water testing. Short clips.

Dataset	What is inside	Track IDs?	Licence	Good for
FishTrack23	About 850k boxes in about 26k tracks (WACV 2024).	Yes	Check	Pre-training the tracker.
Melops	24,578 images of 9,861 individual wrasse, PIT-tag ground truth (2026).	Individual IDs (images)	Check	Testing individual fish re-ID.
FathomNet	Large marine image archive.	No (images only)	Not checked	Detection only, not tracking.

Missing on the internet: no public dataset we found has **hour-long** underwater video with the same fish labelled across **multi-minute disappearances** plus event labels. We will have to make a small one ourselves.

6. What we can build today

Using only the tools above, without new research, we can build a working system:

Part	Built with	Status
Video reading with exact timestamps	PyAV / FFmpeg	Ready
Fish detector	YOLO fine-tuned on SFISHTRACK	Ready (needs a GPU for training)
Continuous tracking across the whole video	BoT-SORT with persistent state	Ready - script in repo: prototype/run_tracking.py (not yet run)
Annotated output video	Re-draw on original frames	Ready (same script)
ID re-linking after short gaps, with 'uncertain'	prototype/identity.py	Prototype done, tests pass on synthetic data
Database of tracks and events	SQLite	Ready
Event rules and simple alerts	Counts, speed, crowding per time window	Ready
Summaries	Qwen3-VL 4B/8B on chosen keyframes	Ready
Text search	FAISS over summaries	Ready
Dashboard	FastAPI + Streamlit	Ready
NEW: Precise fish masks + fixing hard cases (fish crossing, overlapping, half hidden)	**SAM 2.1** (tiny/small), started from YOLO boxes	Ready - run only when needed (see next page)
NEW: Clean fish crops for re-ID (no coral or water background)	**SAM 2.1** masks	Ready
NEW: Fast labelling of test videos	**SAM 2.1**: draw one box, it follows the fish	Ready
NEW: Following one important target for a long time (turtle, ROV inspection object)	**SAM2Long** (no training needed)	Ready to test

This alone is a **useful product**, but it is mostly **engineering** - joining existing tools. To make it a research project that is truly ours, we must solve the problems in section 7.

Where SAM 2 fits in our system

SAM 2 (Segment Anything Model 2, Meta) is a **mask tracker**: you mark an object once (click or box) and it follows that object pixel by pixel through the video, using a memory of earlier frames. It is strong on general video benchmarks (we have not yet tested it on our underwater footage), but it **does not find new objects by itself** and gets slower with every extra object it follows. So it is a **helper** next to YOLO + BoT-SORT, not a replacement.

#	Where we use SAM 2	How	Why it helps
1	**Hard tracking moments** - two fish cross, overlap or one is half hidden	When BoT-SORT or the identity memory says 'uncertain', give SAM 2 the boxes from just before the problem and let it follow both fish through it	Masks separate touching fish much better than boxes, so fewer ID swaps
2	**Clean crops for re-identification** (STIM)	Cut the fish out with its mask before making the appearance embedding	The embedding describes the fish, not the coral or water behind it - better matching after gaps
3	**Labelling our long-gap test set**	Draw a box on one frame, SAM 2 follows the fish; a person only fixes mistakes	Labelling becomes much faster; the missing test data (P9) becomes affordable
4	**One important target for a long time** - a turtle, a diver, an inspected pipe	SAM2Long, which adds a memory tree to SAM 2 for long videos	Keeps the target over long sequences with less drifting
5	**Better measurements for temporal patterns**	Use mask area and shape instead of boxes	More accurate fish size, group density and zone occupancy over time

Where we should NOT use SAM 2

- As the **main tracker for many fish on every frame** - it is too slow and memory grows with each object.
- To **find new fish** - it needs a prompt; YOLO (or SAM 3 with a text prompt like 'fish') must find them first.
- To **recognise a fish that left the view and came back minutes later** - that is the job of the identity memory and STIM.

Simple rule: YOLO finds, BoT-SORT follows, **SAM 2 fixes the hard moments and draws precise outlines**, the identity memory remembers.

7. Problems we must solve (this makes it ours)

These are the parts that nobody has fully solved for long underwater video. Solving even one of them well is the project's own contribution.

#	Problem	Why existing tools fail	Our direction
P1	**Recognise the same fish after minutes away**, even when others look identical	Trackers remember for seconds. Normal ReID learns 'species', not 'this fish'.	Self-taught identity learning from the video itself + global linking + 'uncertain' when not sure (section 8).
P2	**Know when NOT to guess**	Most trackers always pick a match, so mistakes look like facts.	Link only when the best match is clearly better than the second best. Report the 'uncertain' rate.
P3	**Chunked processing without forgetting**	Independent clips reset IDs at every boundary.	Clips are work units only; tracker state, identity memory and summaries carry across.
P4	**VLM that does not invent things** in murky water	VLMs are not tested underwater and can name fish or events that are not there.	Give the VLM only tracker-confirmed facts; reject any output that names unknown IDs.
P5	**Long video understanding on a small GPU**	Sending every frame to a VLM is far too expensive.	Event-triggered keyframes + duplicate removal + summaries in levels (clip, 10 min, video).
P6	**Real anomaly vs tracking error**	A 'fish disappeared' alert is often just the tracker losing it.	Health-gated alerts: ignore alerts when visibility is low or the tracker is creating many new IDs.
P7	**Underwater image quality**	Enhancement looks nicer to people but may not help the detector.	Do not enhance by default; test raw vs enhanced. Use a visibility score instead.
P8	**Temporal patterns from noisy, uncertain tracks**	Pattern statistics (revisits, cycles) silently assume every ID is correct; one ID swap creates fake patterns.	Uncertainty-aware pattern mining: carry the 'uncertain' links into the statistics and report ranges (section 4).
P9	**No long-gap test data**	Public datasets are short or have no identities across long gaps.	Create a small labelled test set (20-40 fish, 3-5 long videos) and a new scoring method.

8. Our new merged approach [Our idea]

Working name: **Self-Taught Identity Memory (STIM)**. It combines four ideas that exist separately on the internet into one system for long underwater video. As far as our searches went, we did not find this combination published, but a full literature search is still needed before claiming it is new.

Idea we borrow	From	What it gives us
Learn identities from the video itself, no labels	new idtracker.ai (eLife 2025)	Free training pairs: same track piece = same fish; two pieces in the same frame = different fish.
Contrastive re-ID for wild fish	Corkwing wrasse study (arXiv 2301.00596)	Shows contrastive learning works on real wild fish.
Link all track pieces together, in rounds	Min-cost flow, GMCP-Tracker, CoNo-Link	Avoids chains of greedy mistakes; uses the whole video.
Strong video features	V-JEPA 2.1	A good starting network for the appearance embedding.
Smart frame selection	VideoBrain	Fewer frames to the VLM: search first, then look closer.

How STIM works

- **Step 1 - Track:** BoT-SORT makes short, reliable track pieces (tracklets).
- **Step 2 - Self-train:** from those pieces, build positive and negative pairs automatically and fine-tune a small embedding head (on V-JEPA 2.1 or a CNN) for this video or site. It learns *these particular fish*.
- **Step 3 - Link in rounds:** join the most certain pieces first, retrain on the bigger groups, link again.
- **Step 4 - Refuse when unsure:** if the best match is not clearly better than the second best, mark 'uncertain'.
- **Step 5 - Ground the VLM:** the VLM describes only confirmed IDs and events; anything else is rejected.
- **Step 6 - Health-gated alerts:** alerts are trusted only when visibility and tracker health are good.

Why this fits our problem

- No identity labels needed - the video teaches itself.
- It gets **better as the video gets longer** (more training pairs) - the opposite of normal trackers.
- The 'uncertain' rule gives an honest, measurable result instead of hidden mistakes.

Known weak points

- idtracker.ai was made for clear lab tanks; open water, turbidity and fish leaving the view will add noise.
- If the tracker merges two fish into one piece, the training pairs become wrong - use only high-confidence pieces.
- For truly identical schooling fish it may still fail; then the right answer is 'uncertain' plus a count estimate.

9. How we prove it works

Experiment	What it does	What it shows
A - Normal tracking	One tracker over the whole video	Baseline without long-term memory
B - Independent clips	Cut into clips, IDs reset each clip	What breaks when you cut naively
C - Memory-aware	Continuous tracker + identity memory	Value of the memory layer
D - STIM	C + self-taught identity learning	Our new contribution

Fair testing rules

- Split by whole video or site, never by frames from the same video.
- Same detector weights in every experiment, so only the tracking/memory part changes.
- Test at gap lengths of 10 seconds, 1 minute and 5 minutes.
- Note: A already keeps state, so C and D will not beat A on very short occlusions. The interesting results are C/D vs B, and C/D vs A on long gaps.

What we measure

Area	Metrics
Detection	mAP50, mAP50-95, precision, recall
Tracking	HOTA, IDF1, ID switches, fragmentation
Long-gap re-ID (new)	Correct links, wrong links, 'uncertain' rate, identity purity, accuracy vs gap length
Understanding	Event time error, event precision/recall, invented-ID rate in VLM text
Temporal patterns (new)	Change points found within +/- tolerance (F1), planted periodic patterns recovered, revisit-interval error, correct pattern descriptions
Alerts	Precision, recall, false alarms per hour, share of alerts caused by tracker errors
Speed	Frames per second, delay, peak GPU memory, VLM calls per hour

Extra experiments

- Backbone swap: normal CNN vs V-JEPA 2.1 features for identity.
- Keyframe policy: every N seconds vs event-triggered vs search-then-look-closer, at equal VLM budget.
- VLM with vs without tracker facts in the prompt (invented-ID rate).
- Raw vs enhanced images for detection and tracking.
- Alerts with vs without the health gate.

10. Build plan, step by step

Phase	Goal	Main tools	Done when
1 Detect	Fine-tune detector, run on video	Ultralytics YOLO, SFISHTRACK	mAP measured on held-out videos
2 Track	Continuous IDs, annotated video	BoT-SORT, run_tracking.py	Output video same length and frame count; IDs do not reset at clip edges
3 Memory	Re-link IDs, store everything	identity.py, SQLite	Planted-gap test works; 'uncertain' cases logged
4 STIM	Self-taught identity learning	PyTorch, V-JEPA 2.1 or CNN	Better long-gap link accuracy than phase 3
5 VLM	Keyframes, summaries in levels	Qwen3-VL 4B/8B quantised	Event times match a hand-made list; zero invented IDs
6 Patterns	Temporal patterns + health-gated alerts	pandas, SciPy, ruptures, HDBSCAN	Planted patterns found; measured false alarms
7 App	Upload, play, search, timeline	FastAPI, Streamlit, FAISS	End-to-end demo on a long video
8 Paper	Experiments A-D, write-up	TrackEval, own long-gap metric	Results tables and figures

Smallest demo that proves the core idea

Phases 1-3 on one 30-60 minute video with a few labelled fish and planted gaps, compared in experiments A, B and C. This already shows that chunked processing can keep identities. STIM, the VLM and alerts come next.

Folder structure

```
docs/ (this plan, research report)
prototype/ identity.py, test_identity.py, run_tracking.py
src/uwvid/ io/ detect/ track/ memory/ events/ vlm/ api/
dashboard/ configs/ eval/ data/ (raw, processed, db, outputs)
```

11. Hardware and costs

Target: one NVIDIA GPU slice with about 20 GB of memory. The numbers below are **rough planning estimates** from model sizes, not measurements.

Component	Rough GPU memory	Plan
YOLO small/medium, FP16	about 1-3 GB	Runs all the time
Small ReID / STIM head	under 1 GB	Runs all the time
V-JEPA 2.1 ViT-L (optional)	about 1-2 GB	Only for identity features
Qwen3-VL 2B/4B	about 5-10 GB	Separate worker, fits alongside detection
Qwen3-VL 8B full precision	about 16 GB just for weights	Does not fit with the rest; use a 4/8-bit version
SAM 2.1 tiny/small	small model (38.9M params for tiny); memory grows with objects and video length	Only on demand for hard cases, crops and labelling
SAM 3	several GB	Only on demand or offline for labelling

- Use mixed precision and a bounded frame queue, so memory stays flat no matter how long the video is.
- Edge (on the ROV/camera): detector + tracker + identity. Cloud: VLM + search. Send only events and keyframes.
- All software listed is free; check the YOLO AGPL licence if this becomes a commercial product (RT-DETR is an alternative).

12. Risks and honest limits

Risk	How likely	What we do about it
Identical schooling fish cannot be told apart after long gaps	High	Report 'uncertain' and counts; measure the 'uncertain' rate honestly
No ready long-gap test data	Certain	Label a small custom set with SAM 3 help in CVAT
VLM makes up details in murky water	Medium-High	Tracker-grounded prompts and ID validation
Self-training learns wrong pairs from tracker mistakes	Medium	Use only high-confidence track pieces; check pair quality
Someone already published the same combination	Unknown	Full literature search before writing the paper
Not enough GPU time	Medium	Small models first, quantisation, run VLM separately
Dataset licences block some uses	Medium	Prefer CC BY data (SFISHTRACK); check each licence

Bottom line: most of this system can be built today from existing open tools. The part that makes it **our own** is: long-gap identity with honest uncertainty (STIM), a tracker-grounded VLM memory, health-gated alerts, and a new way to test all of this on long underwater video.

13. Sources

Checked by web search in November 2026. Licences and numbers should be confirmed on each official page before use.

1. Ultralytics YOLO26 docs - https://docs.ultralytics.com/

2. YOLO26 announcement - https://www.ultralytics.com/blog/ultralytics-yolo26-the-new-standard-for-edge-first-vision-ai

3. MFT25 and SU-T paper (arXiv 2507.06400) - https://arxiv.org/html/2507.06400v3

4. SU-T project page - https://vranlee.github.io/SU-T/

5. SFISHTRACK record (UIB) - https://portalinvestigacio.uib.es/documentos/6a29a0822b7a0802a525921a

6. FishTrack23 (WACV 2024) - https://openaccess.thecvf.com/content/WACV2024/html/Dawkins_FishTrack23_An_Ensemble_Underwater_Dataset_for_Multi-Object_Tracking_WACV_2024_paper.html

7. BrackishMOT (arXiv 2302.10645) - https://arxiv.org/pdf/2302.10645

8. Melops dataset (Scientific Data 2026) - https://iris.univr.it/retrieve/fe930e5b-2ab4-4e29-a004-01f0efc87f87/s41597-026-07045-1.pdf

9. SAM 3 official repository - https://github.com/facebookresearch/sam3

10. SAM 2 / SAM 2.1 official repository - https://github.com/facebookresearch/sam2

11. SAM2Long (ICCV 2025) - https://openaccess.thecvf.com/content/ICCV2025/html/Ding_SAM2Long_Enhancing_SAM_2_for_Long_Video_Segmentation_with_a_ICCV_2025_paper.html

12. SAM 3 in Transformers - https://huggingface.co/docs/transformers/model_doc/sam3_video

13. V-JEPA 2 / 2.1 repository - https://github.com/facebookresearch/vjepa2

14. V-JEPA 2.1 paper (arXiv 2603.14482) - https://arxiv.org/abs/2603.14482

15. Qwen3-VL technical report - https://arxiv.org/pdf/2511.21631v2

16. Qwen3-VL repository - https://github.com/QwenLM/Qwen3-VL

17. InternVideo2.5 paper - https://arxiv.org/html/2501.12386v2

18. InternVideo3 paper (code release unconfirmed) - https://arxiv.org/pdf/2606.12195

19. VideoBrain (arXiv 2602.04094) - https://arxiv.org/abs/2602.04094

20. New idtracker.ai (eLife reviewed preprint) - https://elifesciences.org/reviewed-preprints/107602v1.pdf

21. Contrastive re-ID of wild fish (arXiv 2301.00596) - https://arxiv.org/abs/2301.00596v1

22. Zhang et al. 2008, network flow with occlusion model - https://www.cvlibs.net/projects/autonomous_vision_survey/literature/Zhang2008CVPR.pdf

23. GMCP-Tracker (ECCV 2012) - https://vilab.epfl.ch/zamir/index_files/GMCP-Tracker_ECCV12.pdf

24. CoNo-Link (arXiv 2312.08951) - https://arxiv.org/html/2312.08951v1

Not re-checked for this document: BoT-SORT, ByteTrack, DeepSORT, RT-DETR, TrackEval, FathomNet, WaterNet / UWCNN, FFmpeg, FAISS, FastAPI, Streamlit.